INSLIB — Roll and Pitch from an IMU

Attitude from gyroscope and accelerometer only, in C

Jan Zwiener

2026

Contents

1 Roll and pitch from an IMU in C **1**
- 1.1 What you get . . . 1
- 1.2 Files to copy . . . 2
- 1.3 Conventions you must match . . . 2
- 1.4 First program . . . 3
- 1.5 The calls you actually use . . . 5
- 1.6 Using your real IMU . . . 6
- 1.7 Gyro bias and standing still . . . 7
- 1.8 Tuning . . . 7
- 1.9 Calibration matters here . . . 9
- 1.10 Checking that it works . . . 9
- 1.11 Where to look next . . . 9

1 Roll and pitch from an IMU in C

This tutorial shows the smallest useful INSLIB setup: **roll and pitch from a gyroscope and an accelerometer.** No GNSS, no magnetometer, no barometer, etc.. It is the right starting point for a balancing robot, a camera gimbal, a tilt sensor, or any project that just needs to know "which way is down".

It uses the attitude filter in `ahrs.h`, in its **ARS mode** (attitude reference system, a "directional gyro"). If you also want position and velocity, continue with c_tutorial.md.

1.1 What you get

Output	Quality
Roll, pitch	Drift-free. The accelerometer measures gravity and prevents long term drift.
Gyroscope bias (x, y, z)	Estimated online and removed continuously.
Roll/pitch 1-sigma	Reported by the filter.

Output	Quality
Yaw	**Not north referenced.** It is only the integrated z gyro and drifts. Without a magnetometer or other heading source there is nothing to prevent drift.

Internally this is a 5-state error-state Kalman filter: roll error, pitch error and three gyro bias components. The gyroscope drives the attitude at the full IMU rate. The accelerometer corrects it a few times per second, and only while it looks like it is measuring gravity (a hard acceleration is recognised and ignored).

1.2 Files to copy

The filter is a handful of plain C files. No build system, no OS calls, no heap. Copy these into your project (flat in one folder is fine, no subdirectories are required):

File	From	Purpose
`ahrs.c`, `ahrs.h`	`src/`	the attitude filter
`geodetic_toolbox.c`, `geodetic_toolbox.h`	`src/`	quaternion and rotation helpers
`sensor_defaults.h`	`src/`	default sensor noise values
`log.h`	`src/`	logging macros, `ahrs.c` includes it
`linalg.c`, `linalg.h`	`KFCore/c/`	small matrix library
`kalman_udu.c`, `kalman_udu.h`	`KFCore/c/`	square-root Kalman update
`miniblas.c`, `miniblas.h`	`KFCore/c/`	the few BLAS routines `linalg` needs

That is 5 `.c` files and 7 `.h` files. Your own code only includes **`ahrs.h`**. Add `log.c` (from `src/`) if you want the optional debug output, see the build section.

You do **not** need `ins.c/ins.h`, `nav_suite.c/nav_suite.h`, `baro_alt.c/baro_alt.h`, `kalman_takasu.*`, and neither the magnetic model (`magnetic_model.*`, `wmm_lut.h`, `wmm_test_vectors.h`) nor anything else for a magnetometer. This holds as long as you compile `ahrs.c` with `-DAHRS_NO_MAG`, which removes the magnetometer code from the filter (see the build section). Without that define `ahrs.c` references the magnetic model and you have to copy `magnetic_model.c`, `magnetic_model.h` and `wmm_lut.h` (a 50 KB data table) as well.

1.3 Conventions you must match

Most "my attitude is wrong" problems are one of these, so check them against your IMU driver first:

- **Body frame is FRD**: x forward, y right, z down. If your sensor is mounted differently, rotate the samples into FRD *before* you call the filter. Quite often IMUs use a different coordinate system.
- **Gyroscope in rad/s**, not deg/s.

- **Accelerometer in m/s², not g.** And it is the *specific force*: what the sensor measures. A level sensor at rest reads **-9.81 on z** (the reaction to gravity points up, z points down). Many IMU datasheets and drivers use the opposite sign or give units of g, convert them.
- **Timestamps in microseconds** (`int64_t`), monotonic, from one clock.
- **Angles in radians.** Roll is positive when the right side goes down, pitch is positive when the nose goes up (Tait-Bryan ZYX, body to NED).

1.4 First program

This is the whole structure you need: level the filter with the first sample, initialise it, then hand every IMU sample to `ahrs_update()`. **The only part that depends on your hardware is `imu_read()`**, put your sensor driver there. The placeholder in the listing just returns a motionless sensor that is tilted by 10° around the x axis, so the program builds and runs as it is, and shows you what to expect.

```c
/* hello_ars.c - roll and pitch from an IMU alone (no GNSS, mag or baro).
 *
 * The structure to copy: level the filter with the first sample, initialise
 * it, then hand every IMU sample to ahrs_update(). Everything that depends
 * on your hardware is in imu_read(). */
#include <stdbool.h>
#include <stdint.h>
#include <stdio.h>

#include "ahrs.h"

#define DEG2RADF(x) ((x) * 0.0174532925f)
#define RAD2DEGF(x) ((x) * 57.2957795f)

/* YOUR PART: replace the body with your sensor driver.
 *
 *   t_us      timestamp of the sample [microseconds], monotonic
 *   gyr_rps   angular rate [rad/s]
 *   acc_mps2  specific force [m/s^2], what the accelerometer measures
 *             (at rest it points UP, so a level sensor reads -9.81 on z)
 *
 * Both vectors in the body frame FRD: x forward, y right, z down.
 * Return false when there is no sample. This placeholder delivers 5 s of a
 * sensor that lies still at 100 Hz, tilted by 10 degrees around the x axis. */
static bool imu_read(int64_t* t_us, float gyr_rps[3], float acc_mps2[3])
{
    static int n = 0;
    if (n >= 500) { return false; }
    *t_us = (int64_t)(++n) * 10000;

    /* TODO: read sensor data here */
    gyr_rps[0] = 0.0f;
    gyr_rps[1] = 0.0f;
```

```c
    gyr_rps[2] = 0.0f;
    acc_mps2[0] = 0.0f;
    acc_mps2[1] = -1.703f; /* -9.81 * sin(10 deg) */
    acc_mps2[2] = -9.661f; /* -9.81 * cos(10 deg) */
    return true;
}

int main(void)
{
    int64_t t_us;
    float   gyr[3], acc[3];

    /* 1. Take the first sample and use it to level the filter. */
    if (!imu_read(&t_us, gyr, acc)) { return 1; }
    float roll0, pitch0;
    ahrs_leveling_from_acc(acc, &roll0, &pitch0);

    /* 2. Configure. Only the initial attitude and its 1-sigma are
     *    mandatory, everything left at 0 takes a default. */
    ahrs_config_t cfg = {0};
    cfg.mode = AHRS_MODE_ARS; /* roll/pitch only, no magnetometer */
    cfg.rpy_init_rad[0] = roll0;
    cfg.rpy_init_rad[1] = pitch0;
    cfg.rpy_init_stddev_rad[0] = DEG2RADF(2.0f);
    cfg.rpy_init_stddev_rad[1] = DEG2RADF(2.0f);

    /* 3. Initialise. The filter is one caller-owned struct, no heap. */
    ahrs_t ahrs;
    if (ahrs_init(&ahrs, &cfg, t_us) != 0) {
        fprintf(stderr, "ahrs_init failed\n");
        return 1;
    }

    /* 4. Feed every IMU sample as it arrives. */
    int n = 0;
    while (imu_read(&t_us, gyr, acc)) {
        ahrs_update(&ahrs, t_us, gyr, acc, NULL /* no mag */,
                    false /* no external stillness flag */);

        /* 5. Read the solution whenever you like, here once per second. */
        if (++n % 100 == 0) {
            float roll, pitch, yaw;
            if (ahrs_get_rpy(&ahrs, &roll, &pitch, &yaw)) {
                printf("t = %4.1f s   roll %6.2f deg   pitch %6.2f deg\n",
                       (double)t_us * 1e-6, (double)RAD2DEGF(roll),
                       (double)RAD2DEGF(pitch));
            }
```

```
        }
    }
    return 0;
}
```

1.4.1 Build and run

From the repository root (the paths follow the repository layout, if you copied the files flat into one folder use `-I.` and drop the prefixes):

```
cc -std=c11 -D_GNU_SOURCE -DAHRS_NO_MAG -DLOG_LEVEL=LOG_LEVEL_NONE \
   -Isrc -IKFCore/c \
   hello_ars.c \
   src/ahrs.c src/geodetic_toolbox.c \
   KFCore/c/linalg.c KFCore/c/kalman_udu.c KFCore/c/miniblas.c \
   -lm -o hello_ars
./hello_ars
```

Expected output (the placeholder sensor sits at 10° roll):

```
t =  1.0 s   roll  10.00 deg   pitch  -0.00 deg
t =  2.0 s   roll  10.00 deg   pitch  -0.00 deg
t =  3.0 s   roll  10.00 deg   pitch  -0.00 deg
t =  4.0 s   roll  10.00 deg   pitch  -0.00 deg
```

Notes on the build line:

- `-DAHRS_NO_MAG` leaves the magnetometer code out of `ahrs.c`, which is what makes the magnetic model files unnecessary. In such a build `ahrs_init()` returns -1 for `AHRS_MODE_AHRS`, and `ahrs_set_position()` and `ahrs_mag_heading()` do not exist. Do not use it if you also want `nav_suite` or the heading mode, they need the magnetometer code.
- `-DLOG_LEVEL=LOG_LEVEL_NONE` switches the library's debug output off completely, so `log.c` is not needed. Leave it out and add `src/log.c` to get a printout of the effective configuration at start (useful while tuning), or install your own sink with `log_set_sink()` (UART, RTT, …). `log.h` is always required, `ahrs.c` includes it.
- `-D_GNU_SOURCE` only makes the standard headers expose `M_PI` to the library sources. It is not a requirement for your own code.
- The library is `-Wall -Wextra` clean and has no dependency other than the C standard library and `libm`.
- On a small microcontroller you can additionally shrink the scratch matrices of the Kalman backend, which default to a size that suits the largest filter of the library, with `-DKALMAN_MAX_STATE_SIZE=6 -DKALMAN_MAX_NOISE_SIZE=6`.

1.5 The calls you actually use

1. **`ahrs_leveling_from_acc(acc, &roll, &pitch)`** - once, from a sample (or better an average of a few) taken while the sensor is held still. It gives the filter the right starting attitude, so it does not have to spend its first seconds converging.
2. **`ahrs_init(&ahrs, &cfg, t_us)`** - once. `t_us` is the timestamp of that first sample. Returns 0 on success, -1 for an invalid configuration (for example a zero initial standard deviation).

3. **`ahrs_update(&ahrs, t_us, gyr, acc, NULL, false)`** - once per IMU sample, as it arrives. It integrates the gyro, runs the Kalman prediction and applies the accelerometer correction when it is due.
4. **`ahrs_get_rpy()`** and friends - read the answer at any time.

The two last arguments of `ahrs_update()` are the magnetometer (`NULL`, ignored in ARS mode anyway) and a *zero rotation* flag, see Gyro bias and standing still.

1.5.1 Reading the result

Function	Returns
`ahrs_get_rpy(&a, &roll, &pitch, &yaw)`	Euler angles in rad. Ignore `yaw` in ARS mode.
`ahrs_get_quaternion(&a, q)`	Body-to-NED quaternion, Hamilton, `q[0] = w`. Use this for 3D math, Euler angles are singular at pitch = +-90°.
`ahrs_get_rpy_stddev(&a, ...)`	1-sigma of roll/pitch in rad (yaw is 0 in ARS mode).
`ahrs_get_bias_gyr(&a, b)`	Estimated gyro bias in rad/s.
`ahrs_get_bias_gyr_stddev(&a, s)`	1-sigma of that bias.

Every getter returns `false` if the filter is not initialised or has become unhealthy. Always check it, as the listing does.

If the filter ever notices that it has lost track (its own attitude uncertainty stays far too large, or a state turns non-finite) it marks itself uninitialised. The getters then return `false`, and you re-do steps 1 and 2 above. In a normal system with a sane IMU this never happens.

1.6 Using your real IMU

Put your driver into `imu_read()` of the listing above. If your code is interrupt or task driven instead of a polling loop, the call stays the same, only the place changes: call `ahrs_update()` once for every new sample, with the sample's timestamp, the gyro in rad/s and the accelerometer in m/s², both in FRD (see Conventions you must match).

A few things the filter does for you, so you do not have to:

- **Bad data is dropped.** A NaN or Inf in the gyro or accelerometer drops that epoch (counted in `ahrs.n_invalid_input`).
- **Time stays honest.** A timestamp that goes backwards is ignored. A gap of 0.2 s or more between two samples (`imu_loss_timeout_sec`) is an IMU loss: whatever the platform turned in it is unknown, so the filter stops (`ahrs_get_rpy()` returns false) instead of carrying on with a wrong attitude. Restart it with `ahrs_init()`, seeding `gyr_bias_init_rps` and `gyr_bias_init_stddev_rps` from `ahrs_get_bias_carry()` so the gyro bias does not have to converge again.
- **Irregular sampling is fine.** The integration uses the *measured* time between samples, not an assumed rate. Use a real hardware timestamp if your IMU provides one, it is more precise than the time at which your code read the sample.

- **Fast loops are cheap.** The covariance prediction and the accelerometer correction are throttled internally (see below), so only the quaternion integration runs at the full IMU rate.

1.6.1 Initial attitude

A single accelerometer sample is noisy, and a vibrating or moving sensor is wrong by the acceleration it experiences. If you can, average 0.5 to 1 s of samples while the sensor is held still, and pass that to `ahrs_leveling_from_acc()`. Then set `rpy_init_stddev_rad` to what you believe the result is good to (a degree or two for a quiet start).

If you cannot guarantee a still start, use a large initial standard deviation (for example 10° to 20°) and the filter pulls itself in from the accelerometer within a few seconds. Zero is rejected, the initial standard deviation is mandatory.

1.7 Gyro bias and standing still

A MEMS gyro never reads exactly zero at rest. This offset (the *bias*) is integrated into the attitude, so an uncorrected 0.5 °/s bias would make the angles walk away at 30° per minute. The filter has the bias as states and removes it:

- **Roll and pitch drift** are directly visible to the accelerometer, so the x and y gyro bias converge while the sensor is tilting around.
- **The z gyro bias** only turns yaw, which gravity cannot see. Roll and pitch do not care, but the (unusable) yaw output does.
- **Standing still** gives the filter a very direct bias measurement. By default it notices on its own when the sensor is stationary (the gyro and accelerometer readings stay quiet for a moment) and then measures all three bias components from the gyro signal. You do not have to do anything for this. If you know from outside that the platform is stationary (wheel encoders, a "motor off" flag), pass `true` as the last `ahrs_update()` argument for those epochs.

A platform that vibrates at standstill (a running engine) may not look still to the automatic detector, which is strict. In that case the filter simply does not use it and relies on the accelerometer for the x/y bias, which still works. If the detector were to arm wrongly on a vehicle that cruises smoothly at constant speed, switch it off with `cfg.auto_zaru_disable = true` and use your own flag instead.

The estimated bias can be given a head start if you stored it from the last run: set `cfg.gyr_bias_init_rps` and `cfg.gyr_bias_init_stddev_rps`.

1.8 Tuning

Everything in `ahrs_config_t` left at 0 selects a default that suits a low-cost consumer MEMS IMU. Start with the defaults. The knobs that matter most for roll/pitch:

Field	Meaning	When to change
`acc_noise_mps2`	How much each accelerometer correction is trusted [m/s²]. This is NOT only the sensor noise: also acts as the margin for non-gravity acceleration.	Larger on a vibrating or accelerating platform (a vehicle, a drone), smaller on a quiet one.
`acc_freq_hz`	Maximum rate of accelerometer corrections [Hz].	Lower if accelerations corrupt the attitude, higher if gyro bias tracking is too slow.
`gravity_diff_penalty`	Extra distrust when the accelerometer magnitude differs from g.	Higher for more resistance during manoeuvres.
`acc_reject_gravity_mps2`	Hard-reject the correction when `abs(norm(acc) - g)` exceeds this [m/s²].	Rarely.
`acc_cutoff_freq_hz`	Low-pass cut-off on the accelerometer [Hz].	Lower on a vibrating platform, at the cost of delay.
`gyr_noise_psd`, `gyr_bias_rw`	Gyro noise density and bias random walk from the datasheet or an Allan variance plot (`tools/allan_variance.py`).	If you know them. A better gyro left at the default is merely fused too cautiously.
`rpy_pred_stddev_rad_sqrts`	Extra attitude process noise [rad/√s] on top of the gyro noise, for what the sensor figure leaves out (scale factor and misalignment under rotation, a non-rigid mount).	Has a default (0 selects it). Make it larger if the attitude stddev looks too optimistic.
`imu_loss_timeout_sec`	Gap in the IMU stream [s] that counts as an IMU loss and stops the filter.	Larger only if your IMU delivers in bursts that far apart.

The basic trade-off: **the gyro is the fast and accurate short-term source, gravity is the slow and drift-free long-term source.** The accelerometer cannot tell gravity from acceleration. Any sustained acceleration (braking in a car, a coordinated turn in an aircraft, a centrifugal force on a rotating platform) looks like a tilt. The filter defends itself by gating on the accelerometer norm and by weighting the corrections lightly, and during a long acceleration it coasts on the gyro. That is the right thing, but it bounds how fast a platform can accelerate before the roll and pitch estimate is only as good as the gyro bias.

1.9 Calibration matters here

This filter has **no accelerometer bias state.** An accelerometer offset of 0.1 m/s² shows up directly as a tilt error of about 0.6°, and the filter cannot tell it from a real tilt. Likewise it has no scale factor or axis misalignment correction. For a precise tilt measurement, calibrate your IMU once and apply the correction to the raw samples *before* calling `ahrs_update()`:

```
/* corrected = M * (raw - fixed_bias), M is a 3x3 matrix, from calibration */
```

The calibration tool `tools/inslib_imu_calib.py` produces exactly these numbers from a short recording without any fixture, see the calibration section of c_tutorial.md.

1.10 Checking that it works

A quick sanity check without any truth reference:

1. Put the sensor flat and still. Roll and pitch should be close to 0° (within the accelerometer's calibration).
2. Tilt it by hand, say 30° around the forward axis. The roll must change to about +30° when the right side goes down. If the sign is wrong, your axes or the sign of your accelerometer are not FRD.
3. Rotate it quickly by 90° and back. The roll or pitch angle must follow without lag and come back to the same value.
4. Leave it alone for a minute. The angles must not creep.

If step 2 or 3 fails, it is almost always a unit, sign or axis problem, see Conventions you must match.

Diagnostic counters live in the `ahrs_t` struct: `n_invalid_input` (samples dropped for NaN/Inf), `n_acc_rejected` (corrections rejected as manoeuvres), `n_downweighted` and `n_restart`. A counter that climbs fast is a hint about what is wrong with the input.

1.11 Where to look next

- `src/ahrs.h` - every option and accessor, documented. This header is the reference for the filter.
- `tests/test_ahrs.c` - small readable usage scenarios.
- `c_tutorial.md` - add GNSS or local positioning to get position and velocity as well.
- AHRS mode (`AHRS_MODE_AHRS`) of the same filter adds a magnetometer for a stable heading. The setup is the same except for the `mag_b` argument and the third initial standard deviation. It needs the build without `-DAHRS_NO_MAG` and the magnetic model files from `src/`.